Secure Software Design - Spring 26

Assignment 3: Secure Implementation & Code Submission

Deadline: 26th April, 2026

Note: Continue with the same project idea you selected in the proposal document. All work must be done in the same groups.

Objective:
This assignment requires students to implement the secure design created in Assignment 2 using secure coding practices. Students must translate their UMLsec-based secure design into a working software prototype while applying secure coding principles, API security, cloud security (if applicable), and other relevant security mechanisms.

The focus of this deliverable is secure implementation, code structure, defensive programming, and security controls integrated into the codebase.

Requirements

1. Implementation Based on UMLsec Design

Students must implement the same system designed in Assignment 2, including:

- Secure architecture
- Security constraints from UMLsec diagrams
- Trust boundaries
- Authentication flows
- Authorization model
- Data protection mechanisms
- Secure communication channels

The implementation must reflect the security controls shown in the UMLsec design

2. Secure Coding Principles (Mandatory)

The code must follow secure coding practices **including but not limited to:**

Input Validation

- Validate all user inputs
- Server-side validation mandatory

- Reject malformed data
- Prevent injection attacks

Authentication Security

- Secure login implementation
- Password hashing (bcrypt/argon2 preferred)
- No plaintext password storage
- Account lockout (optional but recommended)

Authorization

- Role-based access control (RBAC) or equivalent
- Privilege checks at API/service level
- Prevent privilege escalation

Error Handling

- No sensitive data in error messages
- Use generic error responses
- Log internal errors securely

Data Protection

- Encryption of sensitive data
- Secure session handling
- Token security (if applicable)

Injection Protection

Protect against:

- SQL Injection
- Command Injection
- XSS
- CSRF (for web apps)

3. API Security (If APIs Are Used)

If the project uses APIs, students must implement:

- Authentication (JWT / OAuth / API keys)
- Authorization checks
- Rate limiting (recommended)
- Input validation for API requests

- Secure headers
- HTTPS usage
- Token expiration handling

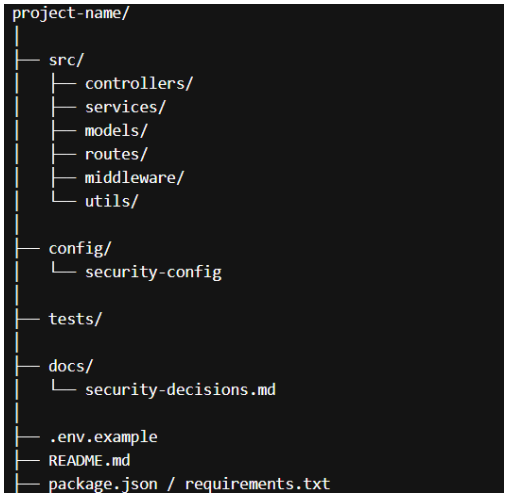
4. Cloud Security (If Cloud is Used)

If deployed on cloud (AWS, Azure, GCP, Firebase, etc.):

- Students must implement:
- Secure environment variables
- No secrets in source code
- IAM / role-based access control
- Secure storage (S3, Blob, etc.)
- HTTPS only communication
- Database access restrictions
- Proper security rules (Firebase, etc.)

5. Folder Structure Requirement

The project must follow a clean and organized folder structure. Example:



Students may adapt structure depending on technology.

6. Code Quality Requirements

Code must include:

- Proper indentation
- Meaningful variable names
- Modular design
- Reusable components

- Separation of concerns
- No hardcoded credentials
- No commented-out insecure code

7. Commenting Requirements

Students must include:

File-level comments

Explain purpose of file

Function-level comments

Explain:

- What the function does
- Security checks performed
- Inputs and outputs

8. README File (Mandatory)

The README must include:

- Project title
- Description
- Security features implemented
- Setup instructions
- Dependencies
- Environment variables
- How to run project

9. Security Features Documentation

Students must submit a short document explaining:

- Authentication method used
- Authorization model
- Encryption used
- API security controls
- Cloud security measures (if applicable)
- Input validation strategy

- Session management

Submission Format

Students must submit:

ZIP File Name:

SSD_Assignment3_GroupName.zip

Contents:

- Source Code
- README.md
- Security Documentation
- Configuration files
- Environment example file